

Asegurar la Calidad del Sw

- Control de calidad del proceso
- " " " " producto
- Prueba de Software

¿Qué es Testing? Proceso destructivo —> encontrar defectos —> Testing exitoso
(cuya presencia se asume)

No asegura calidad, ayuda a controlarlo :)

El testing no puede asegurar ni calidad en el sw ni software de calidad

La calidad del producto depende de las tareas realizadas durante todo el proceso

Niveles de Prueba

• Pruebas Unitarias - Testing Unitario

Se prueban componentes, de forma individual, tras su construcción. Los errores se suelen reparar tan pronto se encuentran. Análisis independiente

• Pruebas de Integración - Testing de Integración

Test orientado a verificar q' los componentes del sistema q' funcionan bien aisladamente, también lo hacen en conjunto.

Estas pruebas deben seguir estrategias incrementales: de arriba hacia abajo } combinar
de abajo hacia arriba }

Los puntos claves de este tipo de prueba:

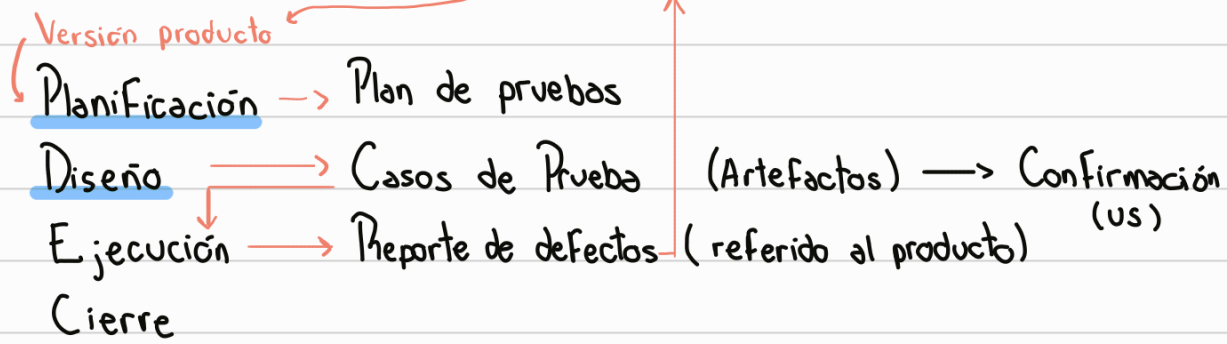
- Conectar de a poco las partes complejas
- Minimizar la necesidad de programas auxiliares

• Pruebas de Sistema - Testing de Sistema



PUD : Req → Analisis → Diseño → Implementación → Prueba → Despliegue *

Actividades del
proceso de
pruebas (testing)



- Lo q' podemos adelantar (50%) → Esfuerzo
 - Lo podemos adelantar desde q' los req' están definidos, comienzo la actividad de planeación y diseño (los CP derivan de los req)
- Los artefactos deben existir sin importar el tipo de gestión.

* Pruebas deben validar y verificar } si no tenemos definidos los CP solo podemos verificar
↓ L> lo q' hace, lo hace bien
Adecuar (hace lo q' debe hacer → más caros para corregir)

Niveles de Testing

- 1) Unitaria - TDD - Implementación (desarrollador)
- 2) Integración - Implementación o Pruebas
- 3) Sistema - Prueba
- 4) Aceptación de usuario - Despliegue

Ambientes de Software (la gente de desarrollo no tiene acceso a Pruebas)

- Desarrollo (pruebas unitarias, pruebas de integración)
- Pruebas (pruebas de sistema)
- Pre-producción (deberían hacerse las pruebas de aceptación)
- Producción (pruebas de aceptación)
- Foto tete :)



Caso de Prueba

situación

descripta detallada y quiere validar y verificar :

→ datos concretos/específicos

- escenario (paso a paso) ← los necesito
- datos de prueba (pre-condiciones)
- resultado esperado (us

necesita

→ Definir en el plan de prueba

¿Cuándo dejar de probar? Nunca, pero hay estrategias

- Estrategias
 - adivinación de errores (x cant. de errores, me acerco → OK)
 - métricas concretas de errores
 - al final del proceso → 2 horas
 - luego de x ciclos
 -

No se asegura calidad con testing, el testing controla la calidad



Ciclos de Prueba ejecución de todos los CP

1er Ciclo se llama ciclo cero → Desarrollo → Ciclo 1

¿Cuántos ciclos es lo ideal? 2 (por cada versión q' entra a Pruebas)

Si o si el cero, encuentra lo máximo de defectos. lo envía a desarrollo, corrige, vuelve a pruebas y todo OK.

Estrategias para cada ciclo de prueba

- 1) Ejecuto el ciclo 0 → Reporte defectos → Desarrollo resuelve —
" " " 1 → Sin regresión (no controla nuevos errores, sino los vistos en el reporte de defectos).
- 2) " " " 1 → corro los CP como si fuera el ciclo 0.
Con regresión

¿Qué probar?

- Requerimientos (Funcionales y no funcionales, se debe probar ambos. Sin embargo los no funcionales son difíciles de probar si los ambientes no son los mismos. Ejemplo CRDS-batata)

Tipos de prueba → foco del producto a probar

- Prueba de Req. (Funcionales)
- Req. Negativa (Funcionales → US (FALLA))
- Pruebas de Proceso de Negocio
- Pruebas Documentación (Manual de usuario, Pl. conf.) → Profe: Funcionales
Libros: no funcionales

Conceptos

- Testing metodológico vs Ad Hoc (no metodológico, no tiene procesos, no sigue una estructura, ni CP)
- Error vs Defecto → Provocan una falla (consecuencia del sw)
 - Defecto: se traslada de una etapa anterior a la actual (Pruebas), es lo q' encuentra Testing
 - Error: no se traslada de una etapa anterior, se descubre en la etapa inicial y allí mismo se corrige (más barato de corregir)

! Testing: proceso de detección de defectos (q' se asume su existencia pq sw es una actividad humano-intensiva)

Lo más costoso del sw es el esfuerzo (50%) el testing debería ser un 30%-50% del costo total del proyecto. La implementación debería llevar un 30%-35%.

- Pair programming → detección inmediata de defectos, ambos deben tener el mismo conocimiento (similar)
- Severidad del defecto (no refiere a lo q' cuesta corregirlo, sino en el impacto en el uso del sistema)
 - Bloqueante
 - Grave
 - Leve
 - Cosmético → interfaz, interacción, ortografía, etc
 - Mejora (no es un defecto, sino una sugerencia)

- Prioridad (alta, media, bajo)